Lctr 21: Additional Load and Store Instructions and Move Instructions

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- Register-offset load and store instructions for different data types
  - 32-bit signed and unsigned
  - 16-bit signed and unsigned
  - 8-bit signed and unsigned
- C pointer operations implemented by the above instructions.
- Pseudo LDR and move instructions. (Can be skipped.)

Demo project:

- Illustration of example problems of various load and store.
- Seeing how `LDR Rd, =num_or_label` works. (Can be skipped.)

C and ASM programming:

- `const` value and `const` pointer. (Can be skipped.)

21.1 Introduction

21.1.1 Immediate-offset load and store instructions

In the previous lecture, we discussed the following immediate-offset load instructions:

Offset type	Instruction
Basic:	`LDR{type} Rv, [Ra{, #os}]`
Pre-indexed:	`LDR{type} Rv, [Ra, #os]!`
Post-indexed:	`LDR{type} Rv, [Ra], #os`

where

- `type` can be `B`, `SB`, `H`, `SH`, and `-`, for unsigned byte, signed byte, unsigned halfword, signed halfword, and word, respectively.
- `Rv` is the register for holding the loaded value, type-casted to 32-bit if needed.
- `Ra` is the register that holds the address of the memory location from where we load the data, which is equivalent to the C pointer. The brackets signifies the address of the memory.
- `#os` is an immediate number for *offset*.

21.1.2 Register-offset load and store instructions

Sometimes, we need to use a variable to express the offset, as we will see the corresponding C code below. This can be done by using the register-offset load (and store) instructions we will discuss below.

21.1.3 Implementation of the pseudo load instruction

We have used a pseudo instruction, `LDR Rd, =num_or_label`, many times in the past. In this lecture, we will see how this instruction is implemented in different situations. To this end, we need to discuss the move instructions as well.

21.1.4 The demo project

21.2 Register-offset load instructions

21.2.1 Basic register-offset load instruction for 32-bit numbers

The register-offset load instructions are just like the above basic immediate-offset instructions. The only difference is that we need to replace `#os` by `Op2`, which is expressed as `Ri, LSL #2`, as shown in the basic form for 32-bit loading instruction below:

```
LDR Rv, [Ra, Ri, LSL #2]
```

which is equivalent to

```
var = *(p32 + i)
```

where we have the following one-to-one mapping:

- `Rv <--> var`
- `Ra <--> p32`
- `Ri <--> i`

Note that we need to shift the value of `Ri` to left by `2` bits because ONE word is saved in FOUR bytes, and the memory is handled in the unit of bytes. This means the actually offset in terms of bytes is `4 * i`.

Of course, we can use `LDR Rv, [Ra, Ri, LSL #3]` to load all the even indexed array elements as the true offset is `8 * i`.

21.2.2 Register-offset load instructions for 16- and 8-bit numbers

We use the same *type* suffixes as we have used in the immediate-offset load instructions to deal with the loading of 16- and 8-bit numbers. They are `H`, `SH`, `B`, and `SB` for unsigned halfword, signed word, unsigned byte, and signed byte, respectively.

- For 16-bit numbers:
  - To load an `uint16_t` number, we use `LDRH Rv, [Ra, Ri, LSL #1]`.
  - To load an `int16_t` number, we use `LDRSH Rv, [Ra, Ri, LSL #1]`.
- For 8-bit numbers:
  - To load an `uint8_t` number, we use `LDRB Rv, [Ra, Ri]`.
  - To load an `int8_t` number, we use `LDRSB Rv, [Ra, Ri]`.

21.2.3 Examples for contrasting different load instructions

	Byte 0	Byte 1
0x2000_0200	0x70	0x72
0x2000_0202	0x74	0x76
0x2000_0204	0x78	0x7A
0x2000_0206	0x7C	0x7E
0x2000_0208	0x80	0x82
0x2000_020A	0x84	0x86
0x2000_020C	0x88	0x8A
0x2000_020E	0x8C	0x8E

Figure 1: Memory map.

Example 1. (Various load instructions.) Given the memory map shown in Figure 1 and the values of two registers for addresses:

- R1 = 0x2000_0204
- R2 = 2

Determine the value of R1 to R6 after running the following code

```
    ldr       r3, [r1, r2, lsl #2]
    ldrsb     r4, [r1, r2, lsl #1]
    ldrsh     r5, [r1, r2, lsl #2]
    ldrsh     r6, [r1, #6]!
    ldrh      r2, [r1, r2, lsl #1]
```

Also indicate which of the instructions are **non-standard** applications of register-offset load operation in the sense of unconventional offset. (For example, if we skip for 8 bytes for 32-bit operation, this is an unconventional offset as we normally skip for 4 bytes.)

Solution.

21.3 Register-offset store instructions

21.3.1 Register-offset store instructions for 32-, 16-, and 8-bit numbers

Following the same principle of immediate-offset store instructions and register-offset load instructions, we have the register-offset store instructions, as listed below:

- For 32-bit numbers: `STR Rv, [Ra, Ri, LSL #2]`.
- For 16-bit numbers: `STRH Rv, [Ra, Ri, LSL #1]`.
- For 8-bit numbers: `STRB Rv, [Ra, Ri]`.

Note that for the last two cases, we only save the least significant halfword and byte in `Rv`, respectively.

21.3.2 Examples for contrasting different store instructions

Example 2. Translate the following C function into assembly function.

```
void mp_store_for_different_types_c(uint32_t *p32, int i, int
    var32) {
     *(p32 + i) = var32;
     *((uint16_t *)p32 + i) = var32;
     *((uint8_t *)p32 + i) = var32;
}
```

Use the following prototype

```
@ void mp_store_for_different_types_s(uint32_t *p32, int i,
    int var32)
```

Solution:

We can see the following from the C code:

- `p32` is a pointer to 32-bit variable. It is passed in to the function by R0.
- `i` is a variable for index. It is passed into the function by R1.
- `var32` is 32-bit variable. It is passed in to the function by R2.

Hence, the translated assembly code is given below:

```
@ void mp_store_for_different_types_s(uint32_t *p32, int i,
    int var32)
mp_store_for_different_types_s:
    str     r2, [r0, r1, lsl #2]
    strh    r2, [r0, r1, lsl #1]
    strb    r2, [r0, r1]
    bx      lr
```

Example 3. Assume we have the following:

- `p32` = 0x2000_0200.
- `i` = 2.
- `var32` = 0xDEAD_BEEF
- Memory is shown in Figure 1.

Find the values of 16 bytes starting from 0x2000_0200 by running the C or ASM functions in Example 2. For better comparison, list the values of the memory before running the code and after the code. List EIGHT bytes in a row, with the one with the lowest address at the left side, as what we see in the display of an IDE.

	Byte 0	Byte 1
0x2000_0200	0x70	0x72
0x2000_0202	0x74	0x76
0x2000_0204	0x78	0x7A
0x2000_0206	0x7C	0x7E
0x2000_0208	0x80	0x82
0x2000_020A	0x84	0x86
0x2000_020C	0x88	0x8A
0x2000_020E	0x8C	0x8E

Figure 2: Memory map.

Solution:

Before running the code, we have the following values, due to the initialization:

```
0x20000200:  0x70 0x72 0x74 0x76 0x78 0x7A 0x7C 0x7E
0x20000208:  0x80 0x82 0x84 0x86 0x88 0x8A 0x8C 0x8E
```

After running the code:

```
0x20000200:  0x   0x   0x   0x   0x   0x   0x   0x
0x20000208:  0x   0x   0x   0x   0x   0x   0x   0x
```

21.4 Examples of using immediate- and register-offset loading instructions

Example 4. Consider the calculation of the L1 norm of a vector $\mathbf{x} = [x_0, x_1, ..., x_{n-1}]$, which is calculated as

$$L_1(\mathbf{x}) = \sum_{i=0}^{n-1} |x_i|,$$

where $|\cdot|$ stands for taking absolute value for the component of the vector, which is expressed in signed integers (or fixed-point real numbers).

Implement the above calculation using the following:

- A C function using the standard `for` loop based on pointer updating.
- ASM implementation of the above pointer-updating C function.
- A C function using the standard `for` loop based on register-offset.
- ASM implementation of the above register-offset C function.

Solution: (See the demo project)

C function with the standard `for` loop based on pointer updating:

```
int mp_array_abs_sum_var_ptr(int *pArr, int n) {
    int sum = 0;
    int i = 0;
    while (i < n) {
        int temp = *pArr++;
        if (temp < 0) {
            temp = -temp;
        }
        sum += temp;
        i++;
    }
    return sum;
}
```

ASM function with the standard `for` loop based on pointer updating:

```
mp_array_abs_sum_var_ptr_s:
    push    {r4, lr}
    ldr     r2, =0   @ sum = 0
    ldr     r3, =0   @ i = 0
```

```

mp_array_abs_sum_var_ptr_s_loop:
    cmp     r3, r1
    bge     mp_array_abs_sum_var_ptr_s_loop_end
    ldr     r4, [r0], #4            @ temp = *pArr++
    cmp     r4, #0                  @ temp < 0 ?
    bge     mp_array_abs_sum_var_ptr_s_loop_sum
    rsb     r4, r4, #0              @ temp = -temp
mp_array_abs_sum_var_ptr_s_loop_sum:
    add     r2, r2, r4              @ sum += temp
    add     r3, r3, #1              @ i++
    b       mp_array_abs_sum_var_ptr_s_loop

mp_array_abs_sum_var_ptr_s_loop_end:
    mov     r0, r2                  @ return sum
    pop     {r4, pc}                @ return sum
```

C function with the standard `for` loop based on register-offset:

```
int mp_array_abs_sum_cst_ptr(int *const pArr, int n) {
    int sum = 0;
    int i = 0;
    while (i < n) {
        int temp = *(pArr + i);
        if (temp < 0) {
            temp = -temp;
        }
        sum += temp;
        i++;
    }
    return sum;
}
```

ASM function with the standard `for` loop based on register-offset

```
mp_array_abs_sum_cst_ptr_s:
    push    {r4, lr}
    ldr     r2, =0
    ldr     r3, =0

mp_array_abs_sum_cst_ptr_s_loop:
    cmp     r3, r1
```

```
    bge     mp_array_abs_sum_cst_ptr_s_loop_end
    ldr     r4, [r0, r3, lsl #2]        @ temp = *(pArr+i)
    cmp     r4, #0
    bge     mp_array_abs_sum_cst_ptr_s_loop_sum
    rsb     r4, r4, #0
mp_array_abs_sum_cst_ptr_s_loop_sum:
    add     r2, r2, r4
    add     r3, r3, #1
    b       mp_array_abs_sum_cst_ptr_s_loop

mp_array_abs_sum_cst_ptr_s_loop_end:
    mov     r0, r2
    pop     {r4, pc}
```

-:-

21.5 Move instructions and the pseudo load instruction

21.5.1 The move instructions

Instruction	Description
`MOV Rt, Op2`	Move the value of Op2 to Rt
`MOVW Rd, #xyz_16`	Move the 16-bit immediate number to the bottom halfword [15:0] and clear the top halfword [31:16]
`MOVT Rd, #xyz_16`	Move the 16-bit immediate number to the top halfword [31:16] without changing the bottom halfword [15:0]

Note that

- The meaning of W in MOVW is word, which used to stand for 16-bit number in the past.
- We will only use the `MOV Rt, Rs` form of the move instruction. For the other forms, we just use `LDR Rt, =expr`, the pseudo load instruction.

21.5.2 The pseudo load instruction

For example, consider the following code

```
mp_various_pseudo_ldr_s:
    push    {r4-r6, lr}         @ Push the related Regs in stack
    initialize_r1_to_r6_local   @ Initialization for stressing the changes

    save_r1_to_r6_local 0   @ Save the registers before operations

    ldr     r1, =0xEE
    ldr     r2, =0xBEEF
    ldr     r3, =0xDEADBEEF
    ldr     r4, =mp_various_pseudo_ldr_s

    save_r1_to_r6_local 24  @ Save the registers after operations
    pop     {r4-r6, pc}     @ Restore the values of Regs and return
```

The pseudo instructions will be translated to

```
    ldr     r1, =0xEE
80003e6:   f04f 01ee   mov.w   r1, #238    ; 0xee
    ldr     r2, =0xBEEF
80003ea:   f64b 62ef   movw    r2, #48879  ; 0xbeef
    ldr     r3, =0xDEADBEEF
80003ee:   4b07    ldr r3, [pc, #28] ; (800040c <mp_various_pseudo_ldr_s+0x54>)
    ldr     r4, =mp_various_pseudo_ldr_s
80003f0:   4c07    ldr r4, [pc, #28] ; (8000410 <mp_various_pseudo_ldr_s+0x58>)
```

At the end of the code file, we have the data saved as:

```
    ldr     r3, =0xDEADBEEF
800040c:   deadbeef    .word   0xdeadbeef
    ldr     r4, =mp_various_pseudo_ldr_s
8000410:   080003b9    .word   0x080003b9
```

21.6 Additional reading

21.6.1 Contrasting post-indexed immediate-offset store instructions with register-offset store instructions

```
void mp_init_mem_block_c1(uint8_t *pArr, int n) {
    for (int i = 0; i < n; i++) {
        *pArr++ = 0x70 + (i << 1);  // We can change the pointer
    }
}
```

```
void mp_init_mem_block_c2(uint8_t *const pArr, int n) {
    for (int i = 0; i < n; i++) {
        // *pArr++ = 0x71 + (i << 1); error: increment of read-only parameter
        *(pArr + i) = 0x71 + (i << 1); // We can use an index variable
    }
}
```

The above two pieces of code can be implemented using the following assembly functions, defined in `ge1s_more_ldr_n_str_n_mov_sfns.s`.

```
@ void mp_init_mem_block_s1(uint8_t *pArr, int n);
@   Input:
@     * pArr -> r0  The pointer to uint8_t
@     * n -> r1     The number of bytes to initialize
@   Output:
@     * None
@   Other registers:
@     * i -> r2, 0x70 -> r3
@     * (0x70 + (i << 1)) -> r4

mp_init_mem_block_s1:
    push    {r4, lr}
    ldr     r2, =0      @ i = 0;
    ldr     r3, =0x70

mpmp_init_mem_block_s1_loop:
    cmp     r2, r1
```

```
    bge     mpmp_init_mem_block_s1_end
    add     r4, r3, r2, lsl #1
    strb    r4, [r0], #1
    add     r2, #1
    b       mpmp_init_mem_block_s1_loop

mpmp_init_mem_block_s1_end:
    pop     {r4, pc}
```

```
@ void mp_init_mem_block_s2(uint8_t *const pArr, int n);
@   Input:
@     * pArr -> r0  The pointer to uint8_t
@     * n -> r1     The number of bytes to initialize
@   Output:
@     * None
@   Other registers:
@     * i -> r2, 0x71 -> r3
@     * (0x71 + (i << 1)) -> r4

mp_init_mem_block_s2:
    push    {r4, lr}
    ldr     r2, =0      @ i = 0;
    ldr     r3, =0x71

mpmp_init_mem_block_s2_loop:
    cmp     r2, r1
    bge     mpmp_init_mem_block_s2_end
    add     r4, r3, r2, lsl #1
    strb    r4, [r0, r2]
    add     r2, #1
    b       mpmp_init_mem_block_s2_loop

mpmp_init_mem_block_s2_end:
    pop     {r4, pc}
```

21.6.2 How to understand `const char *const str`